

Checklist

Multiple-Team Manager Checklist

1. Own the role

- ☐ Accept that my job is no longer primarily to write production code.
- ☐ Stay technically credible through reviews, debugging, incidents, and design conversations.
- ☐ Keep at least one protected block of time each week for creative or technical work.
- ☐ Make sure I am building managers, tech leads, and future leaders, not just shipping work.

2. Prioritize what matters

- ☐ Separate **important** work from merely **urgent** work.
- ☐ Reserve time every week for important but not urgent work.
- ☐ Review my calendar and remove low-value meetings.
- ☐ Avoid treating email, chat, and scheduled meetings as automatically important.
- ☐ Make sure every recurring group meeting has a clear purpose and expected outcomes.

3. Manage time deliberately

- ☐ Protect uninterrupted time for strategic thinking.
- ☐ Follow up on key deliverables without becoming a bottleneck.
- ☐ Regularly ask: “Am I spending too much time on urgent but low-value work?”
- ☐ Make room for hiring plans, team health, roadmaps, and organizational development.

4. Delegate on purpose

- ☐ Delegate **simple + frequent** tasks.

- ☐ Handle **simple + infrequent** tasks myself when explaining them would take longer.
- ☐ Use **complex + infrequent** tasks as coaching opportunities for rising leaders.
- ☐ Delegate **complex + frequent** work to develop stronger team ownership.
- ☐ Identify tasks only I can do, and teach the rest.

5. Build leaders under me

- ☐ Give tech leads and emerging managers real responsibility, not just helper tasks.
- ☐ Coach people to handle planning, coordination, and follow-through themselves.
- ☐ Use delegation to grow capability, not just reduce my workload.
- ☐ Check whether my teams could keep operating well if I stepped away for a week.

6. Watch for warning signs

- ☐ Notice disengagement, sudden withdrawal, or unusual silence from team members.
- ☐ Watch for tech leads who report “everything is fine” but skip 1:1s or provide vague updates.
- ☐ Pay attention when meetings feel lifeless or one-sided.
- ☐ Notice teams whose priorities change constantly with no stable goals.
- ☐ Look for teams that are overly narrow, fragmented, or resistant to learning outside their area.

7. Say no like a leader

- ☐ Use “yes, and” instead of reflexive rejection.
- ☐ Clarify tradeoffs: what slips if we say yes?
- ☐ Create policies when I find myself saying “no” repeatedly.
- ☐ Ask “help me say yes” questions to test weak proposals.
- ☐ Use time and budget constraints honestly.
- ☐ Avoid vague “not right now” answers unless I truly intend to revisit later.
- ☐ Deliver clear no’s quickly when needed.

8. Measure team health beyond output

- ☐ Track **release frequency**.
- ☐ Track **frequency of code check-ins / small-batch progress**.
- ☐ Track **incident frequency** and operational load.

- ☐ Look for process friction that slows delivery.
- ☐ Check whether engineers have what they need to do good work.
- ☐ Check whether people have enough focus time to build rather than just react.

9. Improve the system of work

- ☐ Push for smaller releases and faster feedback loops.
- ☐ Reduce manual steps, handoffs, and slow approvals.
- ☐ Improve tooling, automation, and developer experience.
- ☐ Prevent incident work from consuming all engineering time.
- ☐ Balance risk reduction with speed; do not let “safety” become a permanent drag.

10. Build the right team culture

- ☐ Avoid “us vs. them” identity-building.
- ☐ Align teams to company purpose, not team superiority.
- ☐ Encourage openness to ideas from outside the team.
- ☐ Focus on being a strong partner across product, design, and peer teams.
- ☐ Treat my peer leaders as my “first team,” not just my direct reports.
- ☐ Build teams that survive leadership changes and reorganizations.

11. Model sustainable leadership

- ☐ Push for efficiency, automation, and simplification instead of longer hours.
- ☐ Avoid sending after-hours signals that normalize overwork.
- ☐ Queue non-urgent weekend/night messages for later when possible.
- ☐ Ask whether faster means smarter, not just more exhausting.
- ☐ Protect myself and my teams from burnout.

12. Review myself regularly

- ☐ What did I spend time on this week that created little value?
- ☐ What important work did I postpone because urgent work crowded it out?
- ☐ What did I delegate recently, and did it help someone grow?
- ☐ Who are my rising leaders, and what am I doing to prepare them?
- ☐ Are releases, code flow, and incident handling improving?
- ☐ Did I send signals this week that reward focus and sustainability, or chaos and overwork?

Weekly manager reset

- ☐ Review team health, risks, and staffing needs.
- ☐ Review the calendar for low-value meetings.
- ☐ Identify one thing to delegate.
- ☐ Identify one important-but-not-urgent item to move forward.
- ☐ Check in with tech leads and future leaders.
- ☐ Remove one friction point from the team's workflow.

Monthly leadership check

- ☐ Are my teams getting more independent?
- ☐ Are my leads and I spending time on the right level of work?
- ☐ Is our culture collaborative, or is it drifting into silo behavior?
- ☐ Are our operating metrics improving?
- ☐ Am I building an organization that can scale without me in the middle of everything?